Error Handling in Async Code

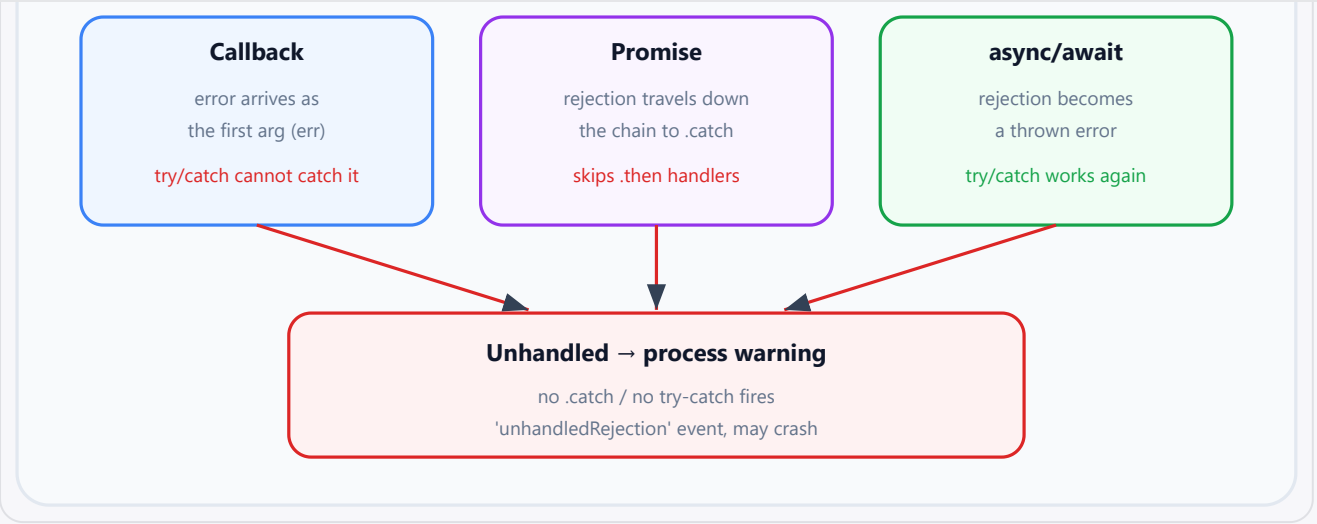
Why it matters

Synchronous error handling is a solved problem: `throw` unwinds the stack, `try/catch` catches it, done. Asynchronous errors are where real Node applications fall over — because an error that fires *after* the stack that scheduled it has unwound cannot be caught the same way, and the three async styles (callbacks, promises, `async/await`) each surface errors in a different place. Get this wrong and errors vanish silently, a single unhandled rejection crashes the whole process, or a failed request hangs forever with no response.

This topic is disproportionately represented in interviews and code reviews because it separates people who can write a happy path from people who can ship a service that stays up. Knowing exactly *where* an error appears in each style, how it propagates, and what happens when nobody handles it is core backend competence.

The concept, simply

The single most important idea: **an async error surfaces at a different place depending on the style**. In a callback, the error is the first argument — `throw` is useless and `try/catch` won't help. In a promise, the error is a *rejection* that travels down the chain to the nearest `.catch`. In `async/await`, the rejection is re-thrown at the `await` point, so ordinary `try/catch` catches it again. Three styles, three landing spots.



Each async style surfaces errors in its own place; anything left unhandled becomes a process-level rejection that can crash Node.

And the safety net matters: if none of those handlers fire, the error becomes a global event — an `uncaughtException` for a thrown sync error, or an `unhandledRejection` for a promise nobody caught. In current Node these are treated as fatal by default and terminate the process.

How it works

Callbacks. Because the callback runs on a fresh stack, the function that scheduled it has already returned. A `throw` inside the callback can't reach that caller — it escapes as an `uncaughtException`. So the convention is to pass the error as the first argument, and every layer must check it and forward it (`if (err) return cb(err)`). Miss one check and the error is silently swallowed.

Promises. Throwing inside a `.then` handler, or returning a rejected promise, rejects the promise that handler returned. That rejection then *skips every subsequent* `.then` and looks downward for a `.catch` (or the rejection handler of a later `.then`). This mirrors how synchronous exceptions skip code and unwind to a `catch` — except it travels *down* the chain rather than *up* the stack. A `.catch` that handles the error and returns a value puts the chain back on the fulfilled track.

async/await. When you `await` a promise that rejects, the engine re-throws the rejection reason as a synchronous exception at that line inside your function. That's the

whole trick: it re-enters the normal exception machinery, so `try/catch/finally` behaves exactly as it does for sync code. An uncaught throw in an `async` function simply rejects the promise that function returned — pushing the problem to *its* caller.

The unhandled path. A promise that rejects with no handler attached (by the time the microtask queue drains) emits `process.on('unhandledRejection')`. A synchronous throw that nothing catches emits `process.on('uncaughtException')`. Both default to crashing. The correct response to `uncaughtException` is to *log and exit*, not to resume — the process is in an unknown state.

Custom Error subclasses. Branching on `err.message.includes('not found')` is fragile — messages get reworded, localised, or wrapped. The robust pattern is to subclass `Error` so callers branch on a *type* or a stable `code` instead of prose. Extend `Error`, call `super(message)`, and set `this.name = this.constructor.name` so the stack header reads correctly. Because a rejection reason travels through the chain by *reference* (it is never copied), `instanceof` keeps working across every `await` and `.catch` between the throw site and the boundary. Give each subclass a machine-readable `code` and an HTTP `status`, and the boundary handler becomes a lookup rather than a pile of string matches.

A small hierarchy of typed errors makes the boundary handler declarative:

JAVASCRIPT

Copy

```
class AppError extends Error {
  constructor(message, { code, status } = {}) {
    super(message);
    this.name = this.constructor.name;    // fixes the stack's error label
    this.code = code ?? 'INTERNAL';
    this.status = status ?? 500;
    Error.captureStackTrace?.(this, this.constructor); // trim our own fra
  }
}
class NotFoundError extends AppError {
  constructor(what) { super(what + ' not found', { code: 'NOT_FOUND', stat
}

async function handler(req, res) {
  try {
    res.json(await loadUser(req.id));     // may throw NotFoundError
```

```
  }
}
```

One built-in subclass is specific to async code: `AggregateError`, the reason `Promise.any` rejects with when *every* input fails. It carries an `.errors` array holding each individual rejection, so you inspect the collection rather than a single reason — the one common case where a rejection reason is naturally a bundle of errors instead of one.

Code examples

The same failure in all three styles, so the landing spot is obvious:

JAVASCRIPT Copy

```
// Callback: error is arg #1. try/catch here would catch NOTHING.
readConfig((err, cfg) => {
  if (err) return console.error('callback error:', err.message);
  use(cfg);
});

// Promise: rejection flows to .catch, skipping the .then above it.
readConfig()
  .then((cfg) => use(cfg))
  .catch((err) => console.error('promise error:', err.message));

// async/await: rejection is re-thrown at the await; try/catch works.
async function run() {
  try {
    const cfg = await readConfig();
    use(cfg);
  } catch (err) {
    console.error('await error:', err.message);
  }
}
```

Propagation: re-throwing (or letting it throw) lets a higher layer decide. Don't catch just to log and swallow — that hides failures:

```
async function loadUser(id) {
  const row = await db.find(id);
  if (!row) throw new Error('user not found: ' + id); // becomes a rejecti
  return row;
}

async function handler(req, res) {
  try {
    const user = await loadUser(req.id);
    res.json(user);
  } catch (err) {
    // Handle ONCE, at the boundary — map to a response.
    res.status(err.message.includes('not found') ? 404 : 500)
       .json({ error: err.message });
  }
}
```

Partial failure with `allSettled`, so one bad item doesn't sink the batch — and a global safety net for anything that slips through:

JAVASCRIPT Copy

```
const results = await Promise.allSettled(ids.map(loadUser));
const users = results.filter((r) => r.status === 'fulfilled').map((r) => r
const failures = results.filter((r) => r.status === 'rejected');
console.log('loaded', users.length, 'failed', failures.length);

// Last-resort net: log and exit — never silently resume.
process.on('unhandledRejection', (reason) => {
  console.error('UNHANDLED REJECTION:', reason);
  process.exit(1);
});
process.on('uncaughtException', (err) => {
  console.error('UNCAUGHT EXCEPTION:', err);
  process.exit(1);
});
```

A subtle propagation trap — `return` vs `return await` inside `try`:

```
async function a() {
  try {
    return await mightReject(); // await → rejection caught here
  } catch (err) {
    return fallback();          // this runs on rejection
  }
}
// With plain 'return mightReject()' the pending promise leaves the try
// block first, so the catch never fires and the caller gets the rejection
```

Common mistakes & gotchas

- **Wrapping async callbacks in `try/catch`.** It catches nothing — the callback runs on a new stack. Check the `err` argument instead.
- **The missing `.catch` / unhandled rejection.** A rejected promise with no handler warns and, by default, crashes current Node. Every chain needs a terminal handler.
- **Swallowing errors.** `catch (e) {}` or a `.catch` that only logs and returns hides real failures and leaves callers thinking things succeeded.
- **Catching too early.** Handling an error deep in a helper and returning `null` forces every caller to guess whether `null` means "empty" or "failed". Prefer to propagate and handle once at the boundary.
- **Rejecting/throwing non-Errors.** `throw 'oops'` or `reject('bad')` loses the stack trace and breaks `err.message`. Always use `Error` objects.
- `return` **vs** `return await` **in a** `try`. Without `await`, the promise escapes the `try` before it can reject, so the local `catch` never runs.
- **Trying to resume after** `uncaughtException`. The process state is corrupt; log and exit (let a supervisor restart), don't continue serving.
- **Sibling rejections in** `Promise.all`. After `all` rejects, other promises keep running; if one later rejects unhandled, you get a separate unhandled rejection.
- **Losing the subclass by cloning the error.** Serialising an error to JSON and rethrowing the plain object strips its prototype, so `instanceof` and `code` checks fail

Best practices

- Always terminate promise chains with `.catch`, or `await` them inside `try/catch`.
- Propagate errors to a single boundary (route handler, job runner) and translate them to responses/retries there — handle once.
- Throw and reject with `Error` subclasses so you keep stacks and can branch on `error.code` or `instanceof`.
- Register `unhandledRejection` and `uncaughtException` handlers that log and **exit**; rely on a process manager to restart.
- Use `allSettled` when partial success is acceptable, so one failure doesn't discard every good result.
- Add cleanup in `finally` (release connections, clear timers) so it runs on both success and failure.
- Enable a linter rule for floating promises and unhandled rejections — catch the bug before production does.

⭐ Interview lens

Q: Why can't `try/catch` catch an error from an async callback?
The callback runs on a fresh stack after the `try` block has already returned. The throw escapes as an `uncaughtException`. That's why the error is passed as the callback's first argument instead.

Q: How does an error propagate through a promise chain?
A throw or rejection skips every subsequent `.then` and travels down to the nearest rejection handler (`.catch` or a `.then`'s second argument), much like an exception unwinding to a `catch`.

It emits `process.on('unhandledRejection')` and, by default, terminates the process with a non-zero exit code. You should attach handlers and, for uncaught cases, log and exit rather than resume.

Q: `unhandledRejection` vs `uncaughtException` — difference?
`uncaughtException` is a synchronous throw nothing caught; `unhandledRejection` is a rejected promise with no handler when the microtask queue drains. Both are last-resort signals; the safe action for both is log and exit.

Q: Should you keep serving after an `uncaughtException`? (trap)
No. The process may be in a corrupt, unknown state. Log, run fast cleanup, and exit; let a supervisor (pm2, systemd, Kubernetes) restart a clean process.

Q: Why prefer propagating an error over catching it deep inside a helper?
Local catches that return `null` force every caller to disambiguate "no data" from "failure" and scatter handling everywhere. Propagating to one boundary centralises the policy (status code, retry, log) and keeps helpers honest.

Cheat-card

- **Callback:** error is arg #1; `try/catch` can't catch it — check & forward.
- **Promise:** rejection flows down to `.catch`, skipping `.then`s.
- **await:** rejection re-throws at the await → normal `try/catch`.
- **Unhandled:** `unhandledRejection` / `uncaughtException` → default crash.
- **Rule:** throw `Error`s, propagate to one boundary, handle once.
- **Safety net:** global handlers should log & `process.exit(1)`, not resume.

QUICK CHECK — no marks, just for you.

- Always as a thrown exception at the call site
- Always via a global handler only
- Always in a `.catch`, regardless of style
- Callback: as the first argument; promise: a rejection to `.catch`; async/await: re-thrown at the `await` for `try/catch`

Check answer

2. What is the correct response to an `uncaughtException` in current Node?

- Log the error and exit, letting a supervisor restart a clean process
- Catch it and resume serving requests as normal
- Ignore it, since Node recovers automatically
- Restart only the affected request

Check answer

3. Why subclass `Error` with a stable `code`/`status` instead of matching on `err.message`?

- Messages are faster to compare than codes
- Message text gets reworded, localised, or wrapped, so branching on a type or stable code via `instanceof` is robust
- You cannot attach properties to a plain Error
- `instanceof` does not work across await boundaries

Previous
Async / Await

Next
EventEmitter

LEARN MORE

ARTICLES & DOCS

try...catch
developer.mozilla.org

Node.js Errors
nodejs.org

VIDEOS



JavaScript Async Await
Web Dev Simplified



Async JS Crash Course - Callbacks, Promises, Async Await
Traversy Media

Hand-picked references for this lesson.